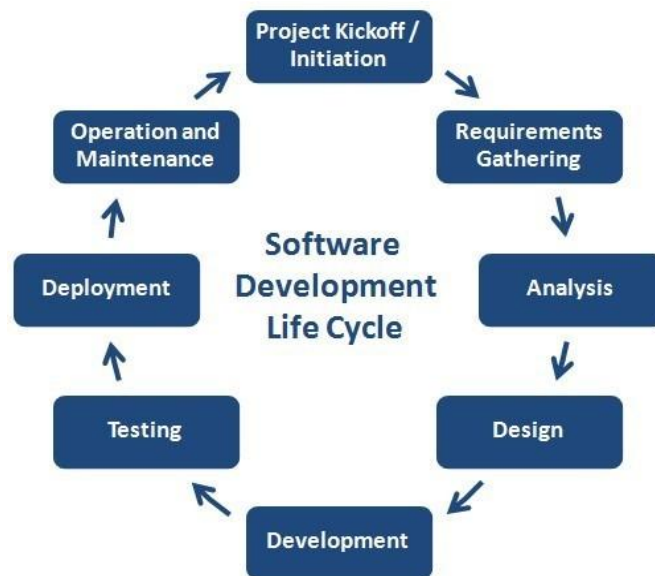


Day 1:

- SDLC (Software Development Life Cycle)



1. Project Kick-off / Initiation
 - a. Purpose:
 - To **start the project officially**
 - Align all stakeholders
 - b. This is the starting phase
 - c. What happens:
 - Project idea is discussed
 - Goals are defined
 - Stakeholders are identified
 - Team introduction
 - Roles & responsibilities assigned
 - Timeline discussion
 - High-level overview of project
 - d. Example:
 - “We are building a flight booking system for client X in 6 months”
2. Requirements Gathering
 - a. Collect what the client wants
 - b. Purpose:
 - To **understand exactly what needs to be built**
 - c. What happens:
 - Collect detailed requirements
 - Functional requirements (features)
 - Non-functional requirements (performance, security, etc.)
 - Client discussions & documentation
 - Business requirements are gathered
 - Functional & non-functional requirements (NFRs) defined
 - d. Example:
 - User can search flights

Performance testing

- System should handle 1000 users (Performance requirement)

3. Analysis

- Understand requirements deeply
- What happens:
 - Feasibility check (technical, cost, time)
 - Identify risks
 - Finalize scope
- Example:
 - Can the system handle peak traffic? What tools are needed?

4. Design

- Plan how to build the system
- What happens:
 - Architecture design (High-level & Low-level)
 - Database design
 - UI/UX planning
- Example:
 - Decide frontend, backend, APIs, DB structure

5. Development

- Actual coding starts
- What happens:
 - Developers write code
 - Build features
- Example:
 - Login page, flight search, booking logic

6. Testing

- Verify the application
- What happens:
 - Functional Testing
 - Performance Testing (The application should be functionally stable for this it's done in pre-prod env)
 - Bug fixing
- Example:
 - Check:
 1. Is booking working?
 2. Is response time acceptable?

7. Deployment

- Release to users
- What happens:
 - Application is deployed to production
 - Made available to users
- Example:
 - App goes live on website

8. Operation & Maintenance

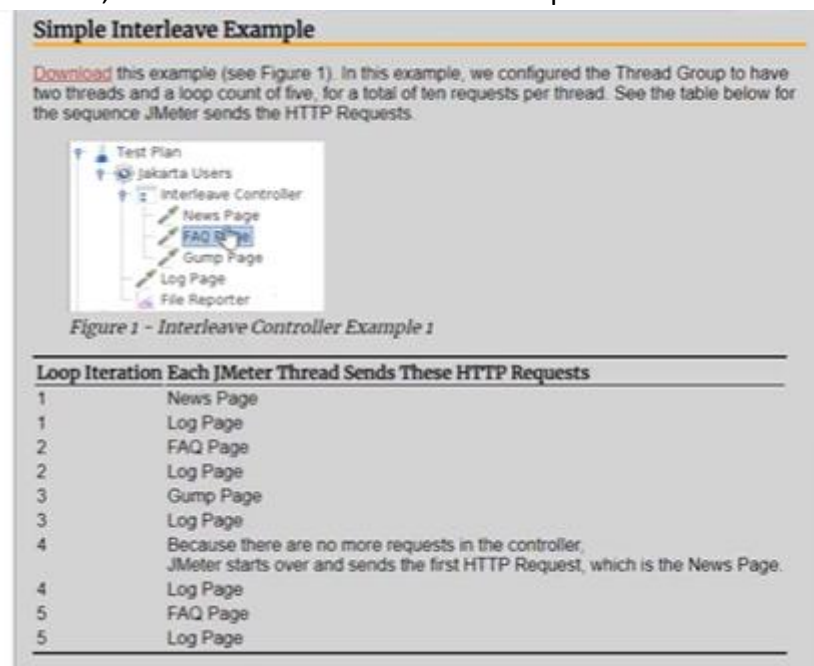
- Support after release
- What happens:
 - Fix bugs
 - Improve performance

Performance testing

- Add new features
- c. Example:
 - Handle user issues, optimize slow APIs
- Important for YOU (Performance Testing Role)
 - You mainly work in:
 - Testing Phase
 - Load testing
 - Stress testing
 - Bottleneck identification
 - Also in:
 - **Requirements phase** → Gather NFRs
 - **Production** → Monitoring performance
- Size of Environments
 - Development env: 4 Core, 16 RAM
 - QA Env: 4, 32 RAM
 - Pre Prod (Performance testing) Env: 8 Core, 124 RAM (Similar to prod env)
 - Prod Enc: 16 Core, 256 RAM
- Jmeter Controller for module wise metrix
- I. WebTour application for sample demo practice
 - WebTour is one application that we can use to do load testing, it is provided by Microsoft
 - Steps: go to google -> search for web tour sample application -> Click on the 1st link (<https://marketplace.opentext.com/appdelivery/content/web-tours-sample-application>)-> click on get it-> Extract the file -> double click on msi-> and install -> extract webtours file -> double click on StartServer.bat -> Allow
 - One cmd line will one where we will get system ip we have to use it to run the application (192.168.1.165 in my case) and “:1080/webTours/” is common for all to access the application “http://192.168.1.165:1080/webTours/”
 - This application will only be accessed in windows
- II. How controllers helps to track performance of modules of application
 - Here each http request is called as a sampler.
 - So collection of may samplers (http request) is called as controller
 - Different Controllers:
 - Recording Controller: Normally to store the recordings of every request as, so when we create an aggregate report for it, it will do it for every request.
 - Transaction Controller: Here we can store the request to perform a single action like login so whatever requests are done to perform login we will store it in that login transaction controller, here in aggregate report it will

give the total time taken by that specific controller to execute all the request present in it.

- **Module Controller:** Here if we want to perform an already recorded action again instead of recording it again or copy pasting its controller, we can use module controller and chain the already recorded request (which is in controller) here, so in the aggregate report it will increase that collections Samples count.
- **Simple Controller:** It is also to store the request in a collection that's all
- **Interleave Controller:** One sampler per iteration is executed top to bottom order, here in this controller we can keep other controllers also



- **Runtime Controller:** It will control the execution of its request/sampler for the given time. Here we will set some time till that time is finished the request will be keep on hitting the server continuously till that time is finished.
- **Random Controller:** Acts same as Interleave Controller but it will pick the child sampler/request randomly (doesn't follows any order)
- **If Controller:** Whatever request/sampler, we keep in this controller we first will give condition in that controller if the condition is correct then it will execute the request inside it else not

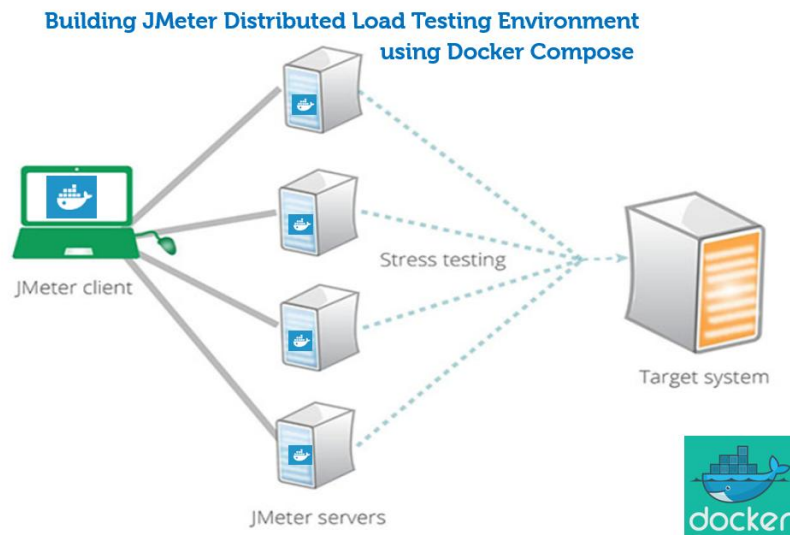
Vars: to read the csv values

Ctx: is the reference variable of JMeterContext class provided by Jmeter, so we can use its methods

Props: It is for the variables create in the test plan

Prev: It's a variable which holds the result of the previous http request/sampler

- JMeter validation in Non GUI Mode(without Jmeter tool)
 - We will do it in cmd
 - Open jmeter bin directory and open it in cmd (always remember open the cmd in admin mode)
 - Write 'jmeter -n' it will activate the jmeter -n file from bin folder, and it also means that we want to run the jmeter script without gui.
 - Then give -t (which means test plan and it specifies from here onwards whatever we will pass is test plan name) the path of the test script .jmx file.
 - Syntax: jmeter -n -t record path
 - To store the result of it we will write -l and path with .jtl extension
 - Syntax: jmeter -n(non gui mode) -t(test paln) testplan path.jml -l(report) report path.jtl
- Importance of blaze meter for cloud execution
 - When we are working with a heavy load that time we go with blazemeter
 - When we don't have enough configurations we go for blazemeter
 - Log in to blaze meter -> Click on Create Test -> JMeter Test -> upload jmeter test (.jmx) extension -> Do the configuration(like no. of users, duration, ramp up time) -> Click on Run test
- Jmeter Distributed modes in running tests in Slave Machine
 - Importance of client server architecture mechanisum
 - There are certain limitations to the number of users you can run, with the CPU, memory and RAM configuration for a single machine
 - Architecture of Jmeter Distributed mode



- We have to create multiple slave (jmeter server) virtual machines in our system.
- Making local system as slave
 - Open jmeter bin folder -> open jmeter properties file -> In that search for "server.rmi.ssl.disable" make it as "true"(by default

it will be false) & remove “#” -> start jmeter-server -> wait for msg saying “created remote object” -> copy the id address of the slave.

- Master machine:
 - In this machine also we have to download jmeter, and the version of jmeter in both the systems(master and slave) should be same
 - In the virtual machine go to jmeter bin folder -> open jmeter.properties file -> In that search for “server.rmi.ssl.disable” make it as “true”(by default it will be false) & remove “#” -> search for remote-host and change the ip address to the slave address.
 - Restart the jmeter in master machine & then run the test on selected nodes
 - Here open the test plan to execute -> if we directly execute it by clicking on that run button it will execute it on the same system but as we want to run it on slave machine we should (click on Run option -> Remote start -> it will show our slaves Ip address select it)
 - Then it will start on the slave machine which we can check on its cmd prompt, and the slave server should be up and running and the result will be shown in the view result tree of master jmeter
 - Also be aware of:
 - The two system must run same Jmeter version
 - The two systems must be in the same subnetwork
 - The two system should be as similar as possible: same OS, same directory tree, etc.
- Monitoring server performance
 - Importance of Server Monitoring
 - We have to monitor the server resources as well. Whenever request is hit to the server it should be capable of handling the load.
 - By doing it we will get to know how cpu is behaving how memory is getting utilized in the server, how many resources it is actually using,
 - So while doing it we should check for both host server and client server (local system)(from client server request is sent to host server).
 - So we will not have host server(actual server), so we will monitor client server(our local system which will send the request) its memory utilization, system utilization.
 - Yourkit profile tool for server monitoring

Performance testing

- Go to google -> search for yourkit java profile -> go to its official website(<https://www.yourkit.com/java/profiler/>) -> Go to download -> download based on your windows -> execute the .exe file -> Evaluate license -> send a mail -> use that license.
- This tool basically listens to all the java profiles, for example apache jmeter will run on JVM so it is monitoring it.
- It will listen to all the applications which run on JVM
- So here whatever actions we will perform on Jmeter it will record all of them.
- To make it listen to the actions we should attach it to the application (click on the application it will connect)